41039 Programming 1

Revision of Week 6: Building reliable classes



Dr. Xuan-Bach Le
le.xuan.bach@hcmut.edu.vn

Faculty of Computer Science and Engineering, HCMUT

September Session 2026

How the revision session works

A recap before the next lecture

Write an answer before the reveal. Explain the state change, rule or contract that makes it follow. These are practice questions, with no new assessment requirements.

Part 1: on your own

- 6 short questions
- About 90 seconds per question
- Java and Python; name the language first

Part 2: teams of 3 or 4

- Draw the model
- Spot the errors
- Predict the output
- Write and test the code

A result needs a reason. A repair needs a counterexample.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

1. A stock invariant

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
Stock item = new Stock(4);
System.out.println(item.take(3));
System.out.println(item.take(2));
System.out.println(item.getStock());
class Stock {
    private int stock;
    Stock(int stock) { this.stock = stock; }
    boolean take(int amount) {
        if (amount <= 0 || amount > stock) return false;
        stock = stock - amount;
        return true;
    }
    int getStock() { return stock; }
}
```

Your answer

What is the complete output? Which line or rule decides it?

1. A stock invariant

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
Stock item = new Stock(4);
System.out.println(item.take(3));
System.out.println(item.take(2));
System.out.println(item.getStock());
class Stock {
    private int stock;
    Stock(int stock) { this.stock = stock; }
    boolean take(int amount) {
        if (amount <= 0 || amount > stock) return false;
        stock = stock - amount;
        return true;
    }
    int getStock() { return stock; }
}
```

Answer

```
true
false
1
```

The first request leaves 1. The second request fails before changing the field, so it stays 1.

2. The same contract in Python

Python 3; commit to an outcome before the reveal.

```
class Stock:
    def __init__(self, stock):
        self._stock = stock
    def take(self, amount):
        if amount <= 0 or amount > self._stock:
            return False
        self._stock -= amount
        return True
    def available(self):
        return self._stock
item = Stock(4)
print(item.take(3))
print(item.take(2))
print(item.available())
```

Your answer

What is the complete output? Which line or rule decides it?

2. The same contract in Python

Python 3; commit to an outcome before the reveal.

```
class Stock:
    def __init__(self, stock):
        self._stock = stock
    def take(self, amount):
        if amount <= 0 or amount > self._stock:
            return False
        self._stock -= amount
        return True
    def available(self):
        return self._stock
item = Stock(4)
print(item.take(3))
print(item.take(2))
print(item.available())
```

Answer

```
True
False
1
```

Both languages implement the same rule. Python prints booleans with initial capitals.

3. One object, two references

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
Stock a = new Stock(5);
Stock b = a;
b.take(2);
System.out.println(a.getStock());
System.out.println(a == b);
class Stock {
    private int stock;
    Stock(int stock) { this.stock = stock; }
    void take(int amount) { stock -= amount; }
    int getStock() { return stock; }
}
```

Your answer

What is the complete output? Which line or rule decides it?

3. One object, two references

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
Stock a = new Stock(5);
Stock b = a;
b.take(2);
System.out.println(a.getStock());
System.out.println(a == b);
class Stock {
    private int stock;
    Stock(int stock) { this.stock = stock; }
    void take(int amount) { stock -= amount; }
    int getStock() { return stock; }
}
```

Answer

```
3
true
```

Assignment makes two names for one stock object. The mutation through b is visible through a.

4. Separate objects, separate state

Python 3; commit to an outcome before the reveal.

```
class Basket:
    def __init__(self):
        self.items = []
a = Basket()
b = Basket()
a.items.append("tea")
print(a.items)
print(b.items)
print(a is b)
```

Your answer

What is the complete output? Which line or rule decides it?

4. Separate objects, separate state

Python 3; commit to an outcome before the reveal.

```
class Basket:
    def __init__(self):
        self.items = []
a = Basket()
b = Basket()
a.items.append("tea")
print(a.items)
print(b.items)
print(a is b)
```

Answer

```
['tea']
[]
False
```

Each constructor call allocates a separate items list; only a receives tea.

5. Class count and instance number

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
Ticket a = new Ticket();
Ticket b = new Ticket();
System.out.println(a.number + " " + b.number + " " +
     Ticket.count);
class Ticket {
    static int count = 0;
    final int number;
    Ticket() { number = ++count; }
}
```

Your answer

What is the complete output? Which line or rule decides it?

5. Class count and instance number

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
Ticket a = new Ticket();
Ticket b = new Ticket();
System.out.println(a.number + " " + b.number + " " +
    Ticket.count);
class Ticket {
    static int count = 0;
    final int number;
    Ticket() { number = ++count; }
}
```

Answer

```
1 2 2
```

The shared counter reaches 2; a keeps its previously assigned number 1.

6. A class attribute can share too much

Python 3; commit to an outcome before the reveal.

```
class Basket:
    items = []
a = Basket()
b = Basket()
a.items.append("tea")
print(b.items)
print(a.items is b.items)
```

Your answer

What is the complete output? Which line or rule decides it?

6. A class attribute can share too much

Python 3; commit to an outcome before the reveal.

```
class Basket:
    items = []
a = Basket()
b = Basket()
a.items.append("tea")
print(b.items)
print(a.items is b.items)
```

Answer

```
['tea']
True
```

Both objects find the class attribute. Move the list into `__init__` to give each basket its own storage.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

Form teams

- Teams of 3 or 4. One shared sheet, one spokesperson.
- For each challenge, record the result and the reason.
- Rotate the spokesperson. Other teams can challenge an explanation.
- The final task has starter files and a Java/Python solution pair.

Discuss first. Commit to an answer. Then compare with the reveal.

Draw it: References inside a composed object

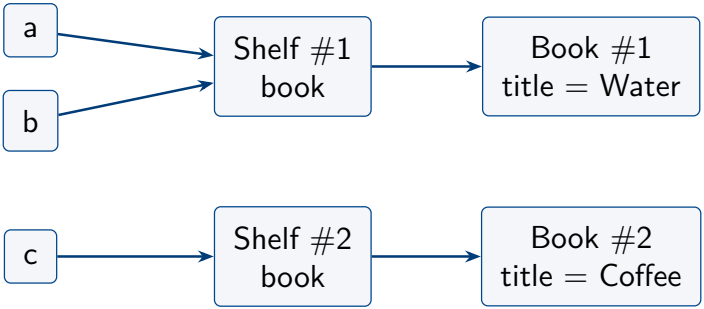
```
Book book=new Book("Tea");
Shelf a=new Shelf(book);
Shelf b=a;
Shelf c=new Shelf(new Book("Coffee"));
b.book.title="Water";
System.out.println(a.book.title+" "+c.book.title);
System.out.println(a==b);
class Book { String title; Book(String t) { title=t; } }
class Shelf { Book book; Shelf(Book b) { book=b; } }
```

Four minutes

Draw the object references after the shown operations. Label the important state. Mark what is shared, what changes, or which path handles failure. Predict every output line.

Course website: Week 6. Complete programs accompany this deck.

Draw it: one answer



Output

```
Water Coffee
true
```

Two shelf objects and two book objects remain. a and b share the first shelf, which holds the renamed book.

Course website: Week 6. Complete programs accompany this deck.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

Spot the errors 1: Java (5 min)

Contract

take accepts a positive amount no larger than stock, preserves state on rejection, and reports success truthfully.

```java
public class Broken {
    public static void main(String[] args)
     {
        Stock s=new Stock(4);
        System.out.println(s.take(0));
        System.out.println(s.take(4));
        System.out.println(s.take(1));
        System.out.println(s.left());
    }
}
class Stock {
    private int count;
    Stock(int count) { this.count=count; }
    boolean take(int n) {
        if(n<0 || n>=count) return false;
        count-=n;
        return false;
    }
    int left() { return count; }
}
```

Spot the errors 1: answers

The repairs

- **L14:** Reject n $<=$ 0, but allow n exactly equal to the available stock.
- **L16:** The successful mutation must return true. The rejection branch still returns false.

✗ Original program

```
false
false
false
3
```

✓ After the repairs

```
false
true
false
0
```

Course website: Week 6. Complete programs accompany this deck.

Spot the errors 2: Python (5 min)

Contract

copy must make an independent basket. Adding milk to b must leave a with tea only.

```
1  class Basket:
2      def __init__(self): self.items=[]
3      def copy(self): return self
4      def add(self,name): self.items.append(
        name)
5      def count(self): print(len(self.items))
```

```
7  a=Basket()
8  a.add("tea")
9  b=a.copy()
10 b.add("milk")
11 print(a.items,b.items)
12 print(a.count())
```

Spot the errors 2: answers

The repairs

- **L3:** Construct a distinct basket and copy the items list.
- **L5:** Return the count instead of printing and returning None.

✗ Original program

```
['tea', 'milk'] ['tea', 'milk']
2
None
```

✓ After the repairs

```
['tea'] ['tea', 'milk']
1
```

Course website: Week 6. Complete programs accompany this deck.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

Predict 1: Shared count, separate counts

```
Counter a=new Counter();
Counter b=new Counter();
a.click(); b.click(); b.click();
Counter c=b;
c.click();
System.out.println(a.mine+" "+b.mine+" "+Counter.total);
System.out.println(c==b);
class Counter {
    static int total=0;
    int mine=0;
    void click() { total++; mine++; }
}
```

Course website: Week 6. Complete programs accompany this deck.

Three minutes

Write every line in order. Explain the first state change, alias, dispatch decision or exception that controls the result. Then propose one change that alters the output.

Predict 1: answer

```
1 3 4
true
```

Why

There are two Counter objects and four clicks. c aliases b rather than constructing another counter.

Course website: Week 6. Complete programs accompany this deck.

Predict 2: Mutation followed by shadowing

```
class Basket:
    items=[]
a,b=Basket(),Basket()
a.items.append("tea")
b.items=["coffee"]
a.items.append("milk")
print(a.items)
print(b.items)
print(Basket.items)
```

Three minutes

Write every line in order. Explain the first state change, alias, dispatch decision or exception that controls the result. Then propose one change that alters the output.

Course website: Week 6. Complete programs accompany this deck.

Predict 2: answer

```
['tea', 'milk']
['coffee']
['tea', 'milk']
```

Why

Appending changes the shared class list. Assigning b.items creates a separate instance attribute for b.

Course website: Week 6. Complete programs accompany this deck.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

Code it: specification

An inventory that preserves its invariant

Read the initial stock, n, then n requests, with each input value on a separate line. Accept positive requests no larger than the remaining stock. Print OK or Refused after each request, then the final stock. Rejecting a request must leave stock unchanged.

Example input

```
5
4
2
4
0
3
```

Expected output

```
OK
Refused
Refused
OK
0
```

Eight minutes. One Java team and one Python team at the keyboard. Other teams produce a boundary test and explain which wrong implementation it would expose.

Code it: Java solution

```java
java.util.Scanner sc = new java.util.Scanner(System.in);
Inventory item = new Inventory(sc.nextInt());
int n = sc.nextInt();
for (int i=0;i<n;i++) System.out.println(item.take(sc.nextInt()) ? "OK" : "Refused");
System.out.println(item.available());
class Inventory {
    private int stock;
    Inventory(int stock) { this.stock = stock; }
    boolean take(int amount) {
        if (amount <= 0 || amount > stock) return false;
        stock -= amount;
        return true;
    }
    int available() { return stock; }
}
```

Course website: Week 6. Complete programs accompany this deck.

Code it: Python solution

```python
class Inventory:
    def __init__(self, stock): self._stock = stock
    def take(self, amount):
        if amount <= 0 or amount > self._stock: return False
        self._stock -= amount
        return True
    def available(self): return self._stock
item = Inventory(int(input()))
n = int(input())
for _ in range(n):
    print("OK" if item.take(int(input())) else "Refused")
print(item.available())
```

Course website: Week 6. Complete programs accompany this deck.

Code it: the tests

A slash separates input lines; (blank) denotes an empty line.

Test 1
`5 / 4 / 2 / 4 / 0 / 3`
Output: `OK / Refused / Refused / OK / 0`

Test 2
`0 / 2 / 1 / -1`
Output: `Refused / Refused / 0`

Test 3
`3 / 0`
Output: `3`

Explain why each test is present, not only whether it passes.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

What you should now be able to explain

- State an invariant and check every mutating path.
- Distinguish copying a reference from copying a model.
- Place per-object mutable data in each instance.
- Use composition and narrow methods to preserve rules.

Return to the matching week page for runnable examples and further practice. Bring one unresolved prediction or counterexample to the next class.

Course website: Week 6. Complete programs accompany this deck.